NO-CODE DEVELOPMENT / N8N PROJECT / SCOPING DELIVERABLE / OCTOBER 2026

YourAnalyst

A personal-finance dashboard with an AI assistant and indicator-based analysis of US stocks

Author: Can Oktay **Target:** Level 3 (advanced)

Stack: n8n (9 workflows), web dashboard with its own chat panel, Postgres, Python analysis API, Yahoo Finance and Frankfurter (FX) APIs, LLM through an OpenAI-compatible endpoint (LM Studio, `qwen2.5-7b-instruct`).

This page set answers the five scoping points. The scope below was validated with the teacher at proposal stage (including the website chat as message trigger) and is the scope that was built.

1 / The need, in one sentence

People who hold cash, stocks and other assets while owing money in several currencies have no single place to see their real net worth and to understand what technical indicators say about their stocks; today they juggle bank apps, a brokerage screen, a spreadsheet of debts and separate charting tools.

YourAnalyst puts this on one screen. The user records accounts, holdings, other assets and debts on the dashboard or in plain English in the chat ("I owe Benjamin EUR 1,000"). The system computes net worth, position values and unrealised profit/loss in one base currency, and analyses US stocks with indicators the user chooses (RSI, MACD, moving-average trend, volume). It is a tracking and research tool: it never trades or moves money.

2 / The trigger, and how to fire it on demand

Main trigger: Webhook POST /webhook/ya-chat from the website chat. The user types in the chat panel next to the dashboard; the page sends `{event_id, text}` with an auth token, and the reply comes back to the same panel. This replaces Telegram/Discord (approved); the excluded n8n Chat Trigger is not used.

Other triggers. Dashboard forms: Webhooks GET /ya-dashboard, POST /ya-record. Signal Lab page: Webhook POST /ya-scan. Valuation refresh: Schedule Triggers (quotes every 10 s, FX every 30 min) plus a Manual Trigger and POST /ya-refresh. Errors: an Error Trigger.

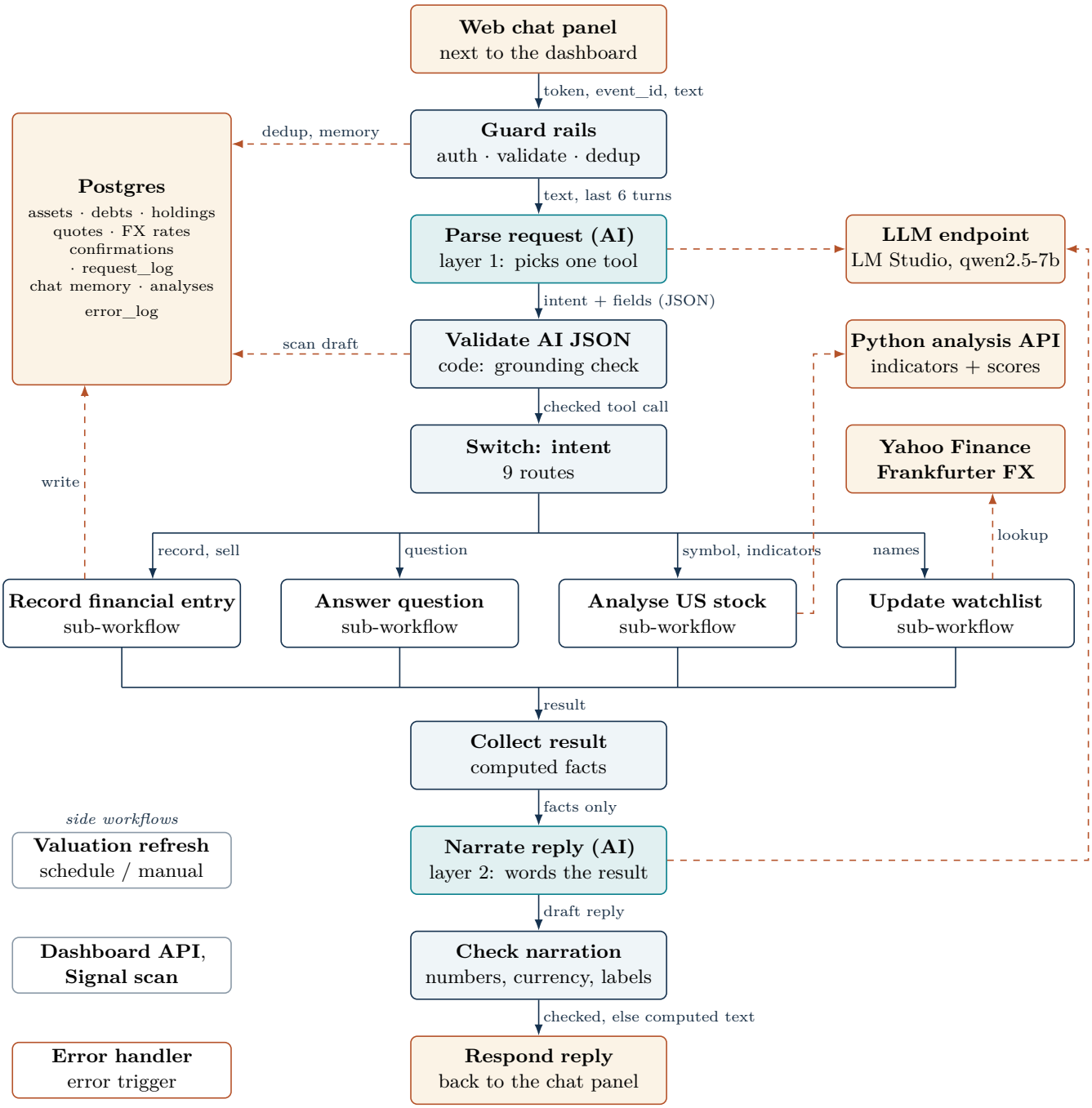
Firing it on demand for testing.

- Type the message in the chat panel, or resend a saved payload from `tests/payloads/` with `tests/send.sh` (curl). Resending a payload with the same `event_id` tests duplicate protection.
- `tests/demo_check.sh --chat` checks containers, n8n, the 9 published workflows, LLM, FX/quote freshness and sends one real message.
- "Run now (manual)" in *YA – Valuation refresh* refreshes prices and FX immediately.

Test input	Expected visible result
I owe Benjamin EUR 1,000	Confirmation question; "yes" saves the debt; net worth updates.
I own 10 AAPL shares at USD 180	Confirmation; saved holding with latest price, value and unrealised P/L.
Analyse AAPL using RSI and MACD	Computed values, data timestamp, explanation tied to those two indicators.
Top five in the demo universe	Ranked stocks with per-indicator contributions and the reason for each score.
I owe him some money	Clarifying question; nothing is written.
Same payload, same `event_id`	The stored reply is replayed; no second row.
Analysis API stopped, then "Analyse MSFT"	3 tries, readable failure, a row in `error_log`.

3 / Architecture diagram

Blocks are n8n workflows or services; each arrow says what travels on it. Numbers are computed only in code and SQL; the AI chooses a tool and words the result.



Solid: the chat request path. Dashed: calls to services and the database. Both AI nodes call the same LLM endpoint (switchable to a hosted model such as OpenRouter by changing one credential). Side workflows: *Valuation refresh* fetches Yahoo quotes and Frankfurter FX into Postgres; *Dashboard API* and *Signal scan* serve the dashboard and the Signal Lab page from Postgres and *Analyse US stock*; *Error handler* is the Error Workflow of all 8 other workflows: it writes `error_log`, marks the request failed and stores a readable reply.

Valuation refresh: Schedule (quotes every 10 s, FX every 30 min) / Manual → tracked symbols and currencies (SQL) → Yahoo / Frankfurter (HTTP) → validate (Code) → upsert. Each quote and rate carries a timestamp; missing prices are marked unavailable and stale FX is labelled.

4 / Target level, and what I cut first

Target: Level 3. Four sub-workflows, an Error Workflow, retries on every HTTP and database node, persisted state (records, pending confirmations, request status, chat memory), duplicate prevention on `event_id`, confirmation before every financial write, and a two-layer AI whose output is checked by code.

Existing foundation, declared upfront. The Python indicator code and the dashboard layout come from my earlier personal project. New for this course: the n8n orchestration, the database, conversational record entry, the AI layers and their checks, and reliable request handling.

Cut line, decided in advance (cut first, in order):

1. Wide scanning (Dow 30, Nasdaq 100, S&P 500, all NYSE), the last feature added. Keep the fixed 20-stock demo universe; if needed, single-stock analysis only.
2. Conversation memory and chat conveniences (multi-turn scan set-up, watchlist by company name). The chat still works one message at a time.
3. Sell flow and what-if questions ("net worth if AAPL reaches 400").
4. Extra asset categories and visual polish (Signal Lab page, heatmap, candle charts).

Never cut: dashboard totals; confirmed debt/asset/holding entry; Postgres persistence; one US-stock analysis with a checked AI explanation; the web chat trigger; sub-workflows; retries, Error Workflow and duplicate prevention.

Level 2 checkpoint. Built first and demonstrable on its own: chat → AI parse → Switch → record or analyse → Postgres → reply. The Level 3 layer (dedup, confirmations, Error Workflow, checks) was added on top.

5 / Minimum technical requirements

Requirement	Where it is used (node names in n8n)
If or Set node	If: authenticated?, If: message valid?, If: new event? (chat router); If: record valid?, If: record found? (dashboard API).
Code node	Validate AI JSON (grounding: every amount must appear in the user's words), Check narration, Compute answer, Validate quotes, Validate FX.
Router or If/Switch	Switch: intent routes record, sell, question, what-if, list, analyse/rank, watchlist, chat, help, clarify; Switch: mode in the sub-workflows.
API or MCP connection	HTTP Request: Analyse API / Rank API (Python analysis service), Fetch quote (Yahoo), Search name (Yahoo), Fetch FX (Frankfurter); LLM through an OpenAI-compatible API.
Message trigger	Webhook POST /ya-chat from my website chat (Telegram/Discord replacement approved by the teacher; the Chat Message trigger is not used).
Database	Postgres nodes: Register request, Load context, Confirm and save, Upsert quotes, Save analysis, Log error + fail request; tables for assets, liabilities, holdings, quotes, FX, confirmations, requests, analyses, errors.
AI node	Basic LLM Chain: Parse request (AI) (intent JSON), Narrate reply (AI), Explain results, Explain answer, Chat reply (AI).
Sub-workflow	Execute Workflow: *Record financial entry*, *Analyse US stock* (also used by *Signal scan*), *Answer question*, *Update watchlist*.
Error handling and retries	Retry On Fail (3 tries, 2 s) on every HTTP and Postgres node; error outputs on Analyse API and Fetch FX; *Error handler* workflow (Error Trigger) set as `errorWorkflow` everywhere; replayed `event_id` returns the stored reply.